

Luke Roberts, Kanchan Kumari, Angelina Murdock
June 2025

Forecasting Home Values with ML

Comparing three ML models for forecasting home values
using Zillow data, covering data, approach, evaluation,
and key results.



Content *overview*

1

Introduction

2

Data Overview

3

Machine Learning Models

4

Model Training &
Evaluation

5

Results & Comparison

6

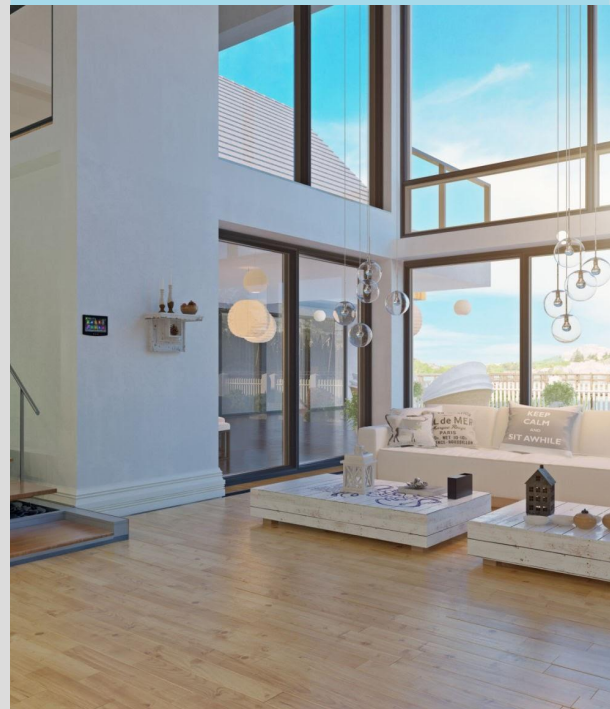
Key Insights

INTRODUCTION

We aim to forecast home values by neighborhood in Denver using time series data. Historic home values are analyzed with three models—**Linear Regression, Keras (Deep Learning Neural Network), and Facebook Prophet**—to identify trends and compare performance.

Our goal is to determine the most accurate model and generate **10-year home value forecasts** for any home type across the city.

Designed for investors, homeowners, and planners, the tool provides **forward-looking, data-driven insights**. Budget is not a constraint—our priority is **maximizing ROI** through accurate and actionable predictions.



Model Development Pipeline

DATA PREPARATION

Clean, aggregate, and format Zillow housing data for time series modeling.

01

MODEL DEVELOPMENT

Build and configure the model (Linear Regression, Keras, or Prophet) using training data.

02

FORECAST GENERATION

Use the trained model to create 10-year home value forecasts by neighborhood.

03

MODEL EVALUATION

Compare forecast accuracy using metrics such as MAE, RMSE, and visual trend analysis.

04

OUR DATA

Data Processing and Exploratory Analysis

01

ZILLOW DATA

Historical home values from Zillow (2000 – 2025) for Denver neighborhoods

02

EDA

Identified trends
Noted missing values
Detected wide variation in home values across neighborhoods

03

DATA CLEANING

Removed null values
Converted data from wide to long format for time series analysis
Grouped data by neighborhood and got yearly/quarterly average home values

Model Comparison

	COMPLEXITY	STRENGTHS	LIMITATIONS	BEST USE CASE
LINEAR REGRESSION	• Low	• Simple, fast, interpretable	• Assumes home value change linearly over time, may miss seasonal or market fluctuations	• Provides a simple baseline forecast of Denver home price trends
KERAS (NEURAL NETWORK)	• High	• Can model complex, nonlinear housing price patterns influenced by market dynamics	• Requires more data and tuning; results are harder to interpret for specific neighborhoods	• Captures intricate relationships in Denver's housing data for improved long-term forecasts
PROPHET	• Medium	• Effectively models seasonal trends and abrupt market changes in Denver's housing prices	• Less customizable for unique local factors, assumes additive seasonal effects	• Forecasting Denver home values with clear seasonality and trend shifts over time

LINEAR REGRESSION MODEL

We built a Linear Regression model to predict average annual home values across Denver neighborhoods.

The model was trained on Zillow housing data and used **Year, Neighborhood, and City** as input features after cleaning and encoding.

We evaluated model performance using Mean Absolute Error (MAE), R^2 , and other regression metrics. Forecasts were generated through the year **2035**.

Part 1

Cleaned and prepared Zillow housing dataset
Created dummy variables for categorical fields
(Neighborhood, City)
Scaled data and handled missing values

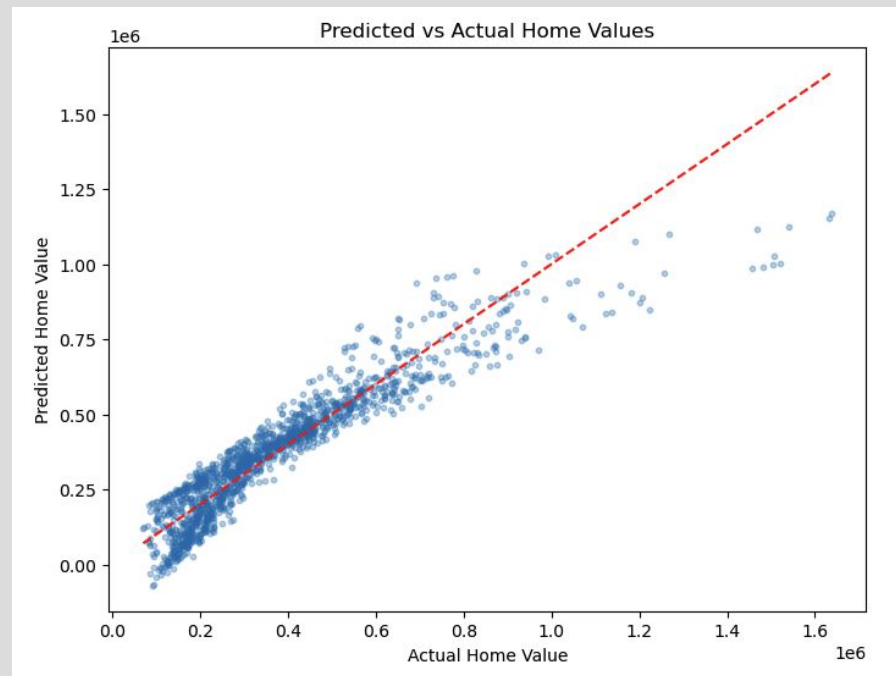
Part 2

Trained Linear Regression model
Split data into train/test sets
Evaluated model using:
• MAE: \$60K
• R^2 : 0.87,
• MAPE: 22.6%

Part 3

Forecasted average home values for 2024–2035
Identified top & bottom neighborhoods for 2035
Saved results as visualizations for Tableau

LINEAR REGRESSION MODEL



This scatter plot shows how well our Linear Regression model performed.

On the x-axis, we have the actual home values, and on the y-axis, we have the values predicted by our model.

The red dashed line represents a perfect prediction — meaning, every dot on this line would be an exact match between predicted and actual.

As you can see, many points follow the trend, but there's a noticeable spread, especially as the home values increase.

This tells us that our model struggles more with higher-priced homes, and that the relationship between features and price may not be strictly linear.

Because of this variation, a non-linear model — such as a neural network or tree-based model — may capture these patterns better."

KERAS MODEL QUARTERLY AND YEARLY

- Compared two Keras models: the **annual model** (more accurate, RMSE $\approx$ \$39K) is ideal for long-term forecasting, while the **quarterly model** (less accurate, RMSE $\approx$ \$82K) captures short-term trends but introduces more volatility—highlighting the tradeoff between prediction stability and data granularity.
- **Model performance metrics reveal trade-offs:** Although the quarterly model captures finer details, its RMSE of $\sim$ \$82K shows diminished accuracy compared to the annual model's $\sim$ \$39K RMSE—demonstrating that more data isn't always better if it adds noise.

Key learning 1

Aggregated (annual) data improves prediction stability and reduces noise.

Key learning 2

Higher frequency data (quarterly) introduces volatility, which challenges model accuracy

Key learning 3

Choosing the right time scale depends on your objective: Use **annual predictions** for long-term investment planning where stability and general trends matter. Use **quarterly predictions** if you're tracking market shifts more frequently, though results may be more volatile.

Keras ML Model (same logic used for yearly and quarterly)

```
# Set features and target
X = df_encoded.drop(['Avg Annual Home Value'], axis=1)
y = df_encoded['Avg Annual Home Value']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

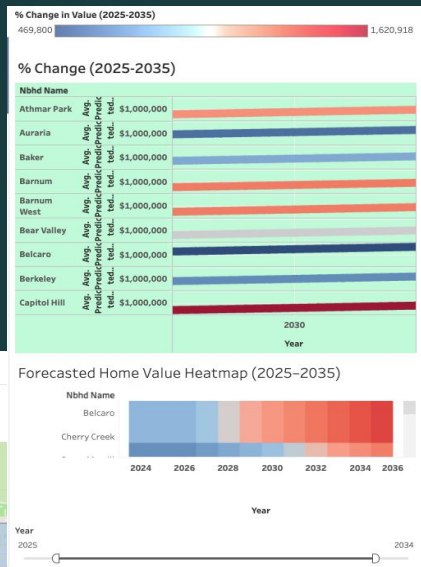
# Build the model
model = Sequential()
model.add(Dense(128, activation='relu', input_dim=X_train_scaled.shape[1]))
model.add(Dense(64, activation='relu'))
model.add(Dense(1)) # output layer for regression

# Compile the model
model.compile(optimizer='adam', loss='mse', metrics=['mae'])

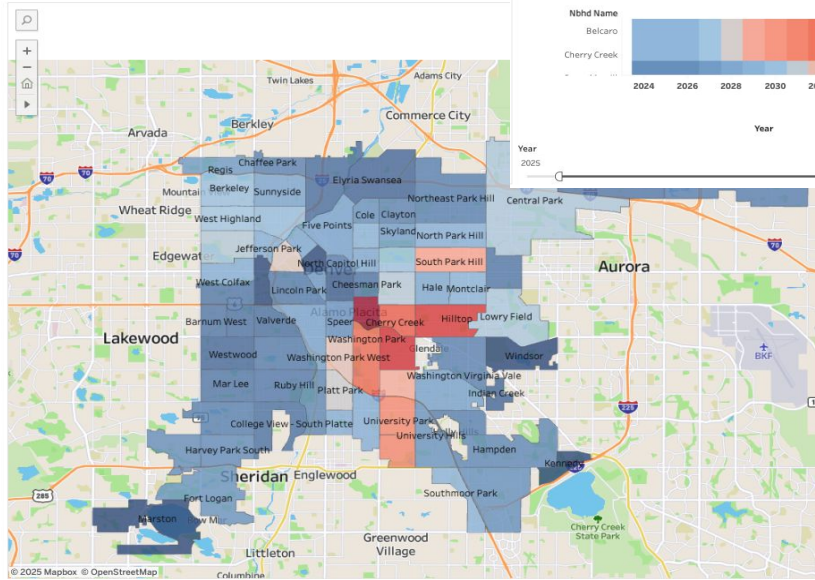
# Train the model
history = model.fit(X_train_scaled, y_train, epochs=100, batch_size=16, validation_split=0.2, verbose=1)
```

- Learns complex, **nonlinear patterns** in housing data across both numeric and encoded features.
- Trained over **100 epochs** to ensure deep learning without premature stopping without overcommitting (memorizing data)
- Uses **20% validation data** to monitor and prevent overfitting.
- Produces **accurate long-range home value forecasts** by balancing model depth and training time.

Tableau DashBoards



10 Year % Map

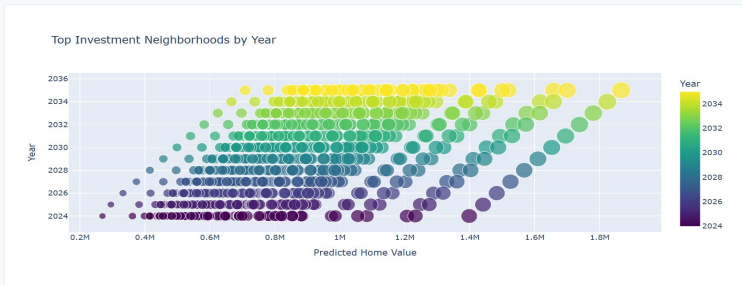


- **Interactive Filtering:** Instantly drill down by neighborhood and price range, to uncover trends.
- **Visual Clarity:** Housing data (e.g., 10-year growth, price forecasts) becomes easy to understand through charts, maps, and tooltips.
- **Geospatial Analysis:** Integrates shapefiles to show neighborhood-level trends on an interactive map.
- **Real-Time Insights:** Filters and dashboards update instantly—ideal for stakeholder presentations or decision-making.
- **Data Integration:** Combines multiple sources (CSV, shapefiles, models) into one cohesive view.

[Tableau Link](#)

Investment Scatter Plot

Start Year: Any End Year: Any Neighborhood: All Min Price: e.g. 500000 Max Price: e.g. 1000000 Filter



Back to Home

Investment Scatter Plot

Start Year: 2025 End Year: 2035 Neighborhood: Capitol Hill Min Price: e.g. 500000 Max Price: e.g. 1000000 Filter



Back to Home

Flask App

Loads future home value predictions from a CSV into a web interface.

Accepts user input (year range, neighborhood, price range) to filter the data dynamically.

Generates an interactive Plotly scatter plot, showing home values by year and neighborhood.

Saves the visualization as an HTML file that can be viewed or embedded easily.

Manual Setup Required: Users must run the Flask app locally or deploy it to a server—it's not instantly shareable like Tableau Public dashboards.

PROPHET MODEL

The model:

- Specifically designed for time series forecasting
- Best for data with:
 - Strong seasonal effects
 - Missing values or outliers
 - Non-linear with trends that change over time
- Well-suited for real estate time series with historical patterns

Final Metrics:

- MSE: \$5,059,110,636.41
- RMSE: \$71,127.43
- MAE: \$64,636.74
- MAPE: 10.24%
- R-squared: 0.9265

PROPHET MODEL

Optimization:

- Forecasted each neighborhood individually, reducing MAPE from 34% to 10%
- Significantly improved accuracy by capturing localized trends

```
# Fit and train model and predict by neighborhood

# Dictionary to hold forecasts for all neighborhoods
all_forecasts = []

# Get list of unique neighborhoods
neighborhoods = df_copy['Neighborhood'].unique()

for neighborhood in neighborhoods:
    # Filter data for the neighborhood and training set only
    neighborhood_data = df_copy[df_copy['Neighborhood'] == neighborhood]
    train_set = neighborhood_data[neighborhood_data['ds'] < forecast_start_date]

    # Initialize and fit Prophet model on training data
    model = Prophet()
    model.fit(train_set[['ds', 'y']])

    # Create future dataframe (extending forecast_periods months after last date)
    future = model.make_future_dataframe(periods=forecast_periods, freq='ME')

    # Predict future values
    forecast = model.predict(future)

    # Filter forecast to only dates >= forecast_start_date
    predicted = forecast[forecast['ds'] >= forecast_start_date][['ds', 'yhat']]

    # Add neighborhood column for identification
    predicted['Neighborhood'] = neighborhood

    # Append to list
    all_forecasts.append(predicted)

# Combine all neighborhood forecasts into a single DataFrame
predicted_df = pd.concat(all_forecasts).reset_index(drop=True)
```

✓ 8.3s

Python

PROPHET MODEL

Optimization:

- Cross-validation to evaluate how far into the future the model performs best
- Helped identify the most reliable forecast horizon for home value predictions

```
# Cross validation
# Evaluates forecast accuracy over time
m = Prophet()
m.fit(train_set)
df_cv = cross_validation(m, initial='730 days', # 2 years of training data before 1st cutoff
                        period='365 days',      # make forecasts every 1 year
                        horizon = '3600 days')   # 10 years forecast horizon
```

	horizon	mse	rmse	mae	mape	mdape	smape	coverage
0	344 days	2.057581e+08	14344.272013	9925.487660	0.053849	0.024904	0.055764	0.350649
1	369 days	2.064875e+08	14369.674633	9985.516532	0.054141	0.025296	0.056048	0.355519
2	370 days	2.061870e+08	14359.214716	9997.886599	0.054143	0.025296	0.056039	0.375000
3	371 days	2.243340e+08	14977.783643	10489.451237	0.057109	0.025509	0.059060	0.367965
4	372 days	2.418437e+08	15551.323448	10955.432959	0.059368	0.026445	0.061546	0.370130
...	...	...	...	...	...	...	...	...
424	3569 days	5.500670e+10	234535.070059	190841.272512	0.596763	0.550748	0.780770	0.454545
425	3597 days	5.532969e+10	235222.630146	192065.448972	0.604955	0.551844	0.785093	0.454545
426	3598 days	5.475305e+10	233993.692152	190398.361889	0.600538	0.550748	0.776763	0.461039
427	3599 days	5.509962e+10	234733.082735	190298.956957	0.600912	0.551844	0.776747	0.467532
428	3600 days	5.608738e+10	236827.742654	192434.415391	0.597781	0.550872	0.782184	0.454545

METRIC COMPARISON

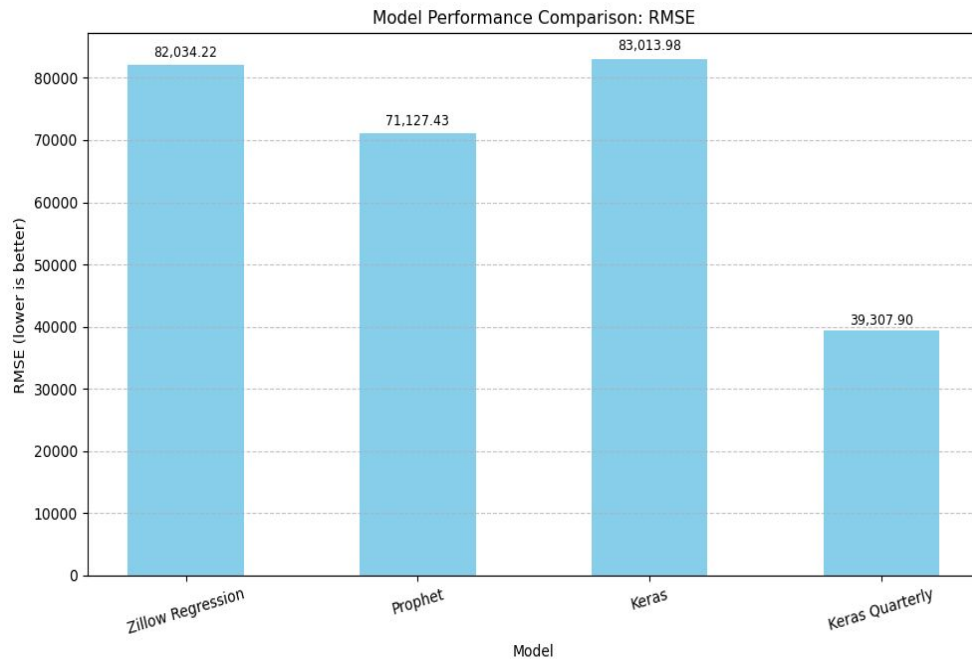
	Linear Regression	Keras	Prophet
R-Squared Shows how well the model fits the data	→ 0.87	→ Yearly: 0.87 → Quarterly: 0.97	→ 0.93
MAE The average of absolute difference between predicted & actual home value	→ \$24,000~	→ Yearly: \$60,000~ → Quarterly: \$23,600~	→ \$65,000 ~
MPE Shows if the model usually over or under estimates. The closer to 0.0 the better the predictions	→ 0.06%	→ Yearly: 1.25% → Quarterly: 0.27%	→ 0.93%
MAPE Mean Absolute Percentage Error	→ 22.61%	→ Yearly: 20.83% → Quarterly: 7.05%	→ 10.24%

FORECASTING DATA COMPARISON

RMSE COMPARISON

Root Mean Squared Error

- Root Mean Squared Error measures how far a models predictions are from the actual values
- More sensitive to large errors
 - Low RMSE = better model performance

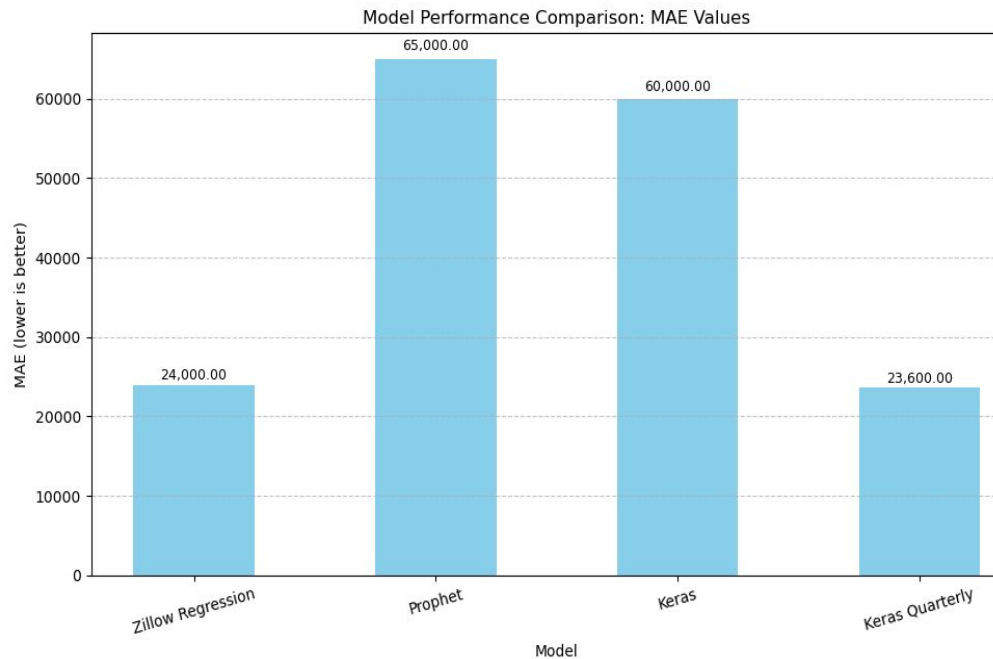


FORECASTING DATA COMPARISON

MAE COMPARISON

Mean Absolute Error

- Compared to RMSE, MAE treats all errors equally and we see a more balanced view across predictions
 - Low MAE = better model performance



FINAL SUMMARY & RECOMMENDATION

Summary:

Forecasted **Denver home values (2024–2035)** using **Linear Regression**, **Keras**, and **Facebook Prophet** models.

- **Linear Regression** showed moderate accuracy ($MAE \approx \$265K$) but less flexibility.
- **Keras** annual model achieved strong accuracy ($RMSE \approx \$39K$), ideal for long-term forecasts; quarterly model was less stable.
- **Facebook Prophet** handled seasonality and trend shifts well, achieving MAPE of 10.24% after neighborhood-level optimization.

Recommendation:

- We recommend **Facebook Prophet** for its balance of accuracy, interpretability, and ability to model complex seasonal patterns and trend changes across neighborhoods.
- For use cases emphasizing long-term stability, **Keras annual model** is a strong alternative.
- Both models outperform linear regression and offer valuable insights for data-driven real estate forecasting.



THANK YOU

Any questions?